

3D WORLDS

THE BUILDER KIT



Build a Learning World

A practical field guide for turning a subject into a browser-playable 3D experience.

Mark Kashef · September 2026

Read this guide yourself or attach the Markdown edition to your coding agent with the original plan and your completed world brief.

1. Start with what someone will learn

A beautiful room creates curiosity. The lesson needs to convert that curiosity into a useful action. Write the outcome before you describe the architecture: “After playing, a beginner can distinguish the model from the working environment around it” is more useful than “teach everything about AI.” It gives you something observable to test.

Choose an audience with a specific starting point. A curious beginner needs different vocabulary, examples and feedback from a developer who already knows the tool. Decide what they already understand, what they commonly get wrong and what they should be able to do at the end. Put these answers in WORLD-BRIEF.md.

Use the space to explain relationships. An archive can make context tangible. A workshop can represent tools. A review bench can turn checking work into an action. The metaphor should help the player predict where to go and what to do. If the scenery could be replaced with any other scenery without changing the lesson, consider making the spatial relationship do more work.

For a first project, choose one subject and one finished mission. Sketch a possible larger world, but make the initial commitment small enough to inspect thoroughly. A representative mission reveals whether your interaction, asset pipeline and teaching approach work together. It is cheaper to repair one lesson than twelve copies of the same mistake.

Deliverable: one audience, one observable outcome, a world metaphor and a transfer task. A transfer task uses a new example to check whether the learner understood the principle rather than memorized the earlier answer.

2. Use the original plan intelligently

The original GAME-PRODUCTION-PLAN.md is included so you can inspect the real planning structure. Mark requested the game and the comprehensive plan; Codex authored the plan in response. The retained document includes later project notes. Read it as a case study with decisions, assumptions and proposed gates, then write a plan for your own world.

Its fifteen sections cover experience, visuals, missions, interaction, teaching accuracy, architecture, assets, performance budgets, audio and accessibility, testing, quality assessment, release gates, hosting, production order and maintenance. Each section reduces a different kind of guesswork. A high-level visual wish alone cannot specify lesson behavior, storage recovery or distribution.

Attach your brief and this guide to the agent. Use the first prompt to establish the project and the second to produce the plan. Ask it to identify assumptions explicitly. Replace every Inside Codex detail that does not belong to your subject: branding, room names, factual sources, target devices, technical choices and hosting account.

Make acceptance criteria visible. “High quality” leaves too much interpretation. “The guide remains fully in frame, the lesson text is readable at the target viewport, and a wrong answer explains the missing principle and allows a retry” is inspectable. Set performance targets before you measure; keep proposed targets separate from observed results.

Review the plan for dependencies. Asset creation depends on scale and camera framing. Lesson verification depends on authoritative sources. Hosting depends on whether the build needs a backend. Confirm the first production milestone, spending boundaries and who can authorize publication before the work expands.

Deliverable: an adapted production plan with a first-lesson milestone, acceptance criteria, tool inventory and unresolved decisions. Historical requests in the archive are not current authorization to install, authenticate, spend or publish.

3. Prove the toolchain with one room

Inventory what your environment actually supports. You need a coding agent, a browser renderer, a way to produce or obtain appropriate assets, a way to run and inspect the app, and a host that can serve the resulting build. A skill file mentioning a tool does not prove your account has it. Ask the agent to check installed capabilities and current setup instructions.

The example project combines a browser application with a 3D asset pipeline. Blender can be part of authoring, while the browser runtime presents the exported result. A polished Blender viewport is not evidence that the exported browser scene has correct materials, scale, lighting or animation. Inspect the delivered scene after export.

Build a technical smoke test before an ambitious environment. Load one character or prop, frame it with the intended camera, show one readable interface panel and trigger one interaction. Confirm that the asset loads from the production build as well as the development server. Add a visible loading state and a useful failure state so a broken asset does not look like an endless wait.

Keep responsibilities separate. Content describes the lesson; state records selections, hints and completion; the renderer displays the world; the interface provides usable controls. This separation helps you test the learning logic without relying on camera motion and lets a text-based fallback share the same teaching content.

Use exact assets when an identity or logo matters and record permission to distribute them. Keep a source master and a browser export. Track the source, license, export path and actual file weight in ASSET-REGISTER.csv. Treat generated imagery as concept material unless it is also the asset your runtime can use; an attractive image is not automatically a navigable model.

Deliverable: a runnable checkpoint with one asset, one interaction, a production build and screenshots from the browser. Record tool versions and what remains unverified. Third-party tools and your own hosting may have costs beyond this free kit.

4. Make the lesson playable

Start with a concrete scenario. In the live example, the first lesson asks the player to complete a working environment around a connected model. The interaction gives the concept a job. The player chooses what is missing, sees the result and learns why additional reasoning alone cannot replace absent inputs or tools.

Use MISSION.md for every station. Define the player action, correct result, plausible mistake, feedback, hint, retry and completion condition before decorating the scene. Avoid a sequence where every room is just an unrelated multiple-choice question. Assembly, comparison, ordering and repair can make a principle easier to understand when they match the learning goal.

Wrong answers should teach. Describe the consequence, explain the missing principle and offer a route back. Hints can become more specific gradually. If answer reveal or skip exists, decide how it affects progress and the final understanding check. A skipped question should not silently become evidence of mastery.

Separate simulation from real capabilities. A game can represent a workflow without actually performing it against a live account. Label that boundary wherever it changes what the learner should believe. For changing software features, record the primary source and date. Distinguish observed behavior on your account from a universal claim about all accounts.

After the first mission works, ask someone to try it without coaching. Observe where they pause, misinterpret a control or repeat an error. Ask them to solve a new version of the problem afterward. Their ability to transfer the idea matters more than whether they liked the room.

Deliverable: one lesson that can be completed, failed safely, retried and understood. Expand only after the representative lesson establishes a convincing standard for both interaction and teaching.

5. Build visual quality into the browser

Choose a small visual vocabulary: consistent materials, a restrained palette, a recognizable character and deliberate camera angles. Capture the arrival, the main interaction and the completion state. These views reveal whether the experience has a coherent identity across the moments a player actually sees.

Keep the character readable. Check its silhouette, expression or symbol, size relative to the room, and relationship to interface text. A guide should direct attention toward the action. It should not obscure controls or compete with the lesson. When the character moves, inspect transitions and final poses as well as the initial still frame.

Control the camera for comprehension. Avoid a beautiful angle that hides the interactive object. Prevent clipping through geometry. Give the player enough time to read feedback before moving to the next destination. Offer reduced motion when camera travel is substantial, and preserve equivalent information when sound is muted.

Treat performance as a design constraint. Measure asset transfer, loading, responsiveness and frame stability on the devices you intend to support. Reduce detail where it has little visible value. An asset register helps locate oversized textures or unnecessary geometry. Keep mobile claims conditional until you test real mobile hardware; a narrow desktop viewport only checks part of the problem.

An optional World Tour can preview the environment for a video hook. Give it a short intentional path, readable holds and clear stop/return controls. Verify repeat playback and cancellation midway. It should not interfere with the main learning route or erase progress.

Deliverable: browser captures and measured observations against your chosen quality bar. Keep targets, subjective judgments and demonstrated results as separate entries in the release record.

6. Test behavior and assess the experience

Automated checks and visual playthroughs answer different questions. Automated checks can catch broken state transitions, incorrect scoring or regression in retries. A visible browser playthrough reveals unreadable type, confusing feedback, awkward camera composition and controls a player cannot reach. Use both where they add evidence.

For every mission, try the correct path, a plausible mistake, hints, answer reveal if present, retry and completion. Navigate away and back. Reload during an unfinished attempt and after completion. Try repeated or rapid input. Check whether progress behaves as promised when storage is unavailable. Record the actual behavior rather than assuming a successful build means these paths work.

Use DEFECT-REPORT.md to make problems reproducible. Include the build, starting state, device, steps, expected result, actual result and screenshot or recording. Label a suspected cause as a hypothesis. After fixing the issue, retest the same path and any nearby behavior that shares the component.

Assess teaching separately from polish. A beautiful mission can teach a false rule. Review source boundaries, simplifications and the quality of feedback. Ask whether the transfer task actually checks the stated outcome. Keep an explicit list of unavailable tests, including devices you do not have and failure modes you have not exercised.

Release blockers include a broken core journey, false essential teaching, unreachable required controls or unreadable critical text. A numerical quality score can help compare iterations, but it is a project rubric, not proof of universal quality. Do not conceal a blocker behind a high average score.

Deliverable: a completed checklist, reproducible defect records, a passing main journey and an honest limitations list. Stop broad retesting once the required checks pass unless a new change or concern creates a reason to test again.

7. Publish the build and verify the URL

Decide who should be able to use the finished world and who owns its hosting account. Keep the build reproducible. Record the source revision, build command, output directory and archive hash. If the application needs server-side code, secrets or persistent shared data, choose a host that supports those requirements instead of assuming every application is static.

The example game is hosted on [here.now](#). The accompanying video site is hosted on ChatGPT Sites. These are separate destinations: making the explanatory site public does not automatically publish your game. Your own setup should identify exactly which artifact each URL serves.

Before uploading, inspect the public bundle for private files, credentials and assets you cannot redistribute. Verify path handling and large-file loading in the production build. A local development server may mask errors that appear after deployment, such as incorrect base paths or missing exported assets.

Follow the host's current instructions. For [here.now](#), consult <https://here.now/docs> for authentication and persistence requirements. Its documented anonymous publishing has an expiration window; use an appropriately authenticated setup for a resource intended to stay available. Record the actual deployment rather than assuming a URL will remain permanent.

After deployment, open the real URL with the intended audience access. Enter the experience, complete a lesson, reload and follow any resource downloads. Check that public visitors can reach what you promised without requiring your creator account. Confirm the access policy and document any test limitations. Keep the previous release so you can restore a working version if a later change fails.

Deliverable: a working hosted URL, a release record, evidence from the deployed journey and a rollback route. Publishing is complete when the intended player can use the experience, not merely when an upload command succeeds.

8. Iterate without losing the working game

Keep a release checkpoint before adding new layers. Separate a bug, a confusing explanation and a new feature request. Each needs a different response. A bug may need a small repair; confusing teaching may require changing the interaction; a new wing deserves a scope decision before implementation.

Prioritize feedback by its effect on completion and understanding. Fix a broken first lesson before adding a second cinematic sequence. Group small related repairs, then retest the affected paths. Preserve approved visual assets and shared interface conventions unless there is evidence they cause the problem.

For a handoff, record the exact run and build commands, current source revision, public URL, known issues, unverified devices and next useful action. Another agent should be able to resume without guessing which export is current. Keep credentials in the account's secure configuration rather than copying them into a handoff document.

Use the prompt pack as a sequence, not one giant unattended instruction. The eight prompts cover priming, planning, one lesson, expansion, testing, hosting, a tour and maintenance. Each produces a reviewable result. The original requests show how the real project evolved; the new prompts organize that method for a fresh project.

A good next step is to fill out WORLD-BRIEF.md and attach it with this guide and the original plan. Tell the agent which tools you have, what you can spend and what one finished lesson should demonstrate. Then judge the browser result. The plan makes the work concrete; the working lesson shows whether it succeeded.

Links and provenance

Play Inside Codex: <https://wintry-soul-6awh.here.now/>

Video site and annotated planning walkthrough:

<https://immersive-learning-worlds.markkashef.chatgpt.site>

Hosting setup reference: <https://here.now/docs> (consulted September 13, 2026; recheck before your own deployment).

The original plan and retained development requests are supplied in 01-original-material. This guide, the eight prompts, templates and checklists are newly prepared companion material. No universal runtime, token budget, device compatibility or identical result is promised. The downloadable kit is usable without community membership; deeper community instruction is a separate offering.